Kaleido

.NET FRAMEWORK OVERVIEW

A .NET framework for business services.

Discover data. Execute steps. Continue.

Run your business code with shared process state.
HTTP for applications. MCP tools for AI.

Reusable business actions. AI-native access.



WHAT KALEIDO ACTUALLY IS

It describes the work. It also runs the work.

A code-first .NET framework for building business services as discoverable queries and executable, stateful steps.

QUERYABLE

Read business information

Publish searchable sources and views.
Example: find a member.

PROCESS

Execute business actions

Run steps and track process progress.
Example: select a provider.

REGISTRY

Describe how to call them

Publish names, inputs, and routes.
Apps and AI can discover the model.

More than a catalog: run the action, record progress, and return what can happen next.

The framework runs within your .NET services; your host supplies identity, business systems, and production storage.

Stop rebuilding the process in every consumer.

Inputs, process rules, and next-step logic should not be redefined in each screen, integration, or AI prompt.

SEPARATE PATHS

Workflow	own adapter + rules
Digital channels	own adapter + rules
AI agents	own adapter + rules

Repeated state and next-step logic.
More places for behavior to drift.

ONE SHARED CONTRACT

Declare and execute the action.
Keep process state in the service.
Return what can happen next.

Each consumer calls the same action and reads the same process guidance.

The goal: less duplicated process plumbing, without replacing the business systems or the channels.

Architecture rationale and intended value; no quantified savings or live integration result is asserted.

The idea needed a working example.

THE COMMUNICATION GAP

Explaining the step concept was not making the real scenarios click.

After several years of explaining the idea, the missing piece was something people could see working.



MAKE THE ACTION VISIBLE

Show what a step needs and what it does.



MAKE THE BEHAVIOR VISIBLE

Show what is required, available, or waiting.



MAKE THE REUSE VISIBLE

Run the same model through different consumers.

Kaleido turns an abstract step concept into an executable model people can understand.

Origin story: founder account, paraphrased; technical framing from the step model.

A step is a business action—not a screen.

Select a provider. Capture answers. Submit a request. Each action runs against the current process state.

INPUT + CONTEXT

Provided values
Process state

ONE BUSINESS ACTION

Declared inputs and rules
Business-owned handler
Recorded outcome

RESULT + NEXT

What happened
How to continue

REQUIRED

A particular step must happen next.

AVAILABLE

The caller chooses among allowed steps.

AWAITING INFORMATION

Specific questions need answers.

Declared relationships + recorded progress + handler outcomes determine how the process may continue.

Conceptual view of the Process contract; the examples are not a fixed PriorAuth sequence.

You write the action. Kaleido supplies the runtime.

Define C# types and metadata, implement the business code, then register them with the framework.

01 DECLARE

The business contract

- Step inputs and descriptions
- Dependencies and availability
- Query sources and views

YOU DEFINE THE SHAPE

02 IMPLEMENT

Your business handler

- Validate business meaning
- Read and write your systems
- Return a result and next direction

YOU OWN THE BUSINESS

03 REGISTER + EXPOSE

The callable surface

- Registry built from declarations
- HTTP query and step endpoints
- MCP tools from the same registry

KALEIDO PUBLISHES THE MODEL

Business logic is written once in the service—not re-created in each channel or prompt.

HTTP and MCP require their host adapters. Registration does not automatically convert arbitrary existing APIs into Kaleido steps.

What happens when someone calls a step?

The caller requests an action. Kaleido applies process rules, invokes the business code, and returns continuation data.

CALLER SENDS — **Step name + input values + process ID (when continuing)**

01 — Load state
Create or reload process progress.

→

02 — Check request
Inputs, completed steps, availability.

→

03 — Run handler
Business meaning and business action.

→

04 — Save state
Record outcome + process progress.

→

05 — Return next
Result, process ID, and next work.

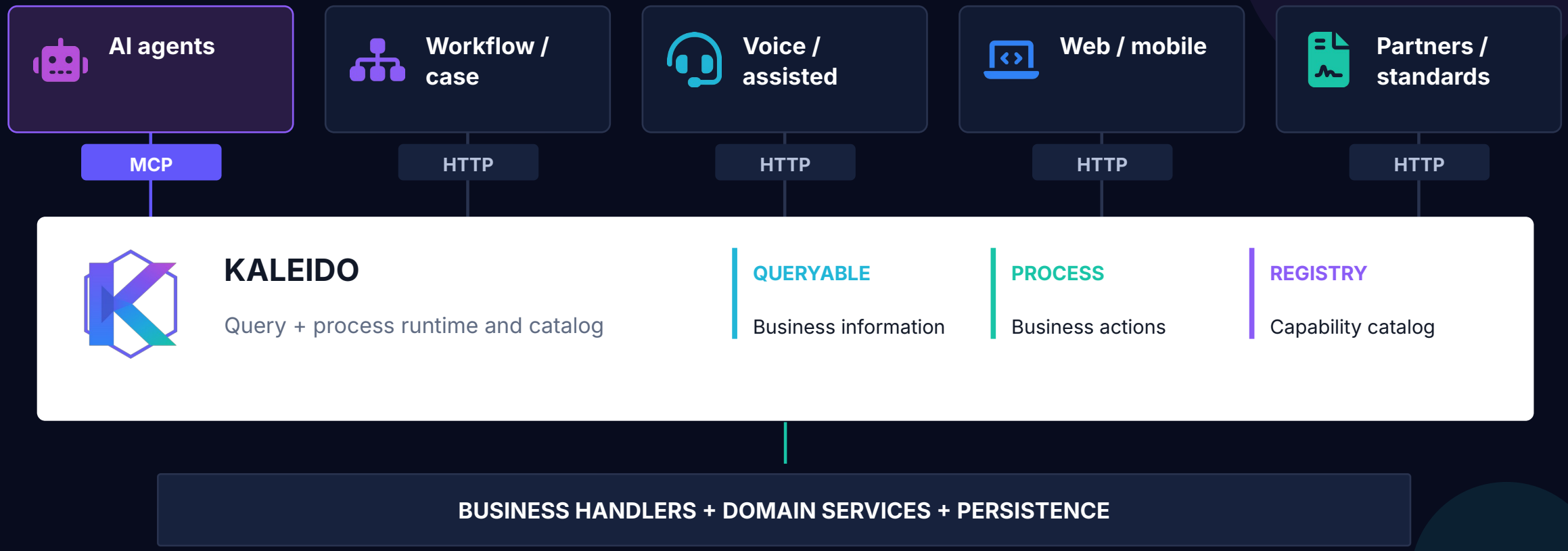
The response explains the next work—it is not just a success code.

- **REQUIRED STEP**
- **AVAILABLE CHOICES**
- **QUESTIONS TO ANSWER**

Conceptual lifecycle. Host authorization is also required; business writes and framework state are not automatically one transaction.

Different interfaces. The same business execution.

AI uses MCP; applications use HTTP. Both reach the same capability contracts and business implementations.



The AI model stays outside Kaleido. Business decisions stay in the application.

Illustrative consumer categories. These are not claims of bundled connectors or vendor certifications.

AI gets tools from the same declared business model.

MCP turns registry metadata into tool schemas; calls reach the same business handlers through the HTTP clients.



01 DISCOVER

Read the tool catalog

Names, descriptions, schemas, and constraints.



02 LOOK UP

Find real business data

Search first; guide the user to a matching record.



03 ACT

Invoke a declared step

Typed arguments enter the Process runtime.



04 ASK

Collect missing answers

Information requests; forms when supported.



05 CONTINUE

Read state and next

Required steps, available choices, and handoffs.

Human choices and business rules remain authoritative. Tool guidance is not authorization.

Implemented source-level capabilities. Form behavior depends on the MCP client; release validation remains open.

Watch one action: selecting a provider.

A query finds candidates. A step changes the request. The response tells the consumer how to continue.

PERSON + AI INTERACTION	BUSINESS CONTRACT
Find a provider Search for provider locations. Show the matching records.	**Queryable + lookup metadata** Return related provider and location IDs from the same query result.
Choose this location The user selects the intended result. Send its IDs with the process ID.	**Process step + business handler** Check process rules and input structure. The handler validates and saves the selection.
Continue the request Show required work or available choices. Ask for service-owned information.	**Outcome + process guidance** Return the result and next-step guidance. The consumer follows the service response.

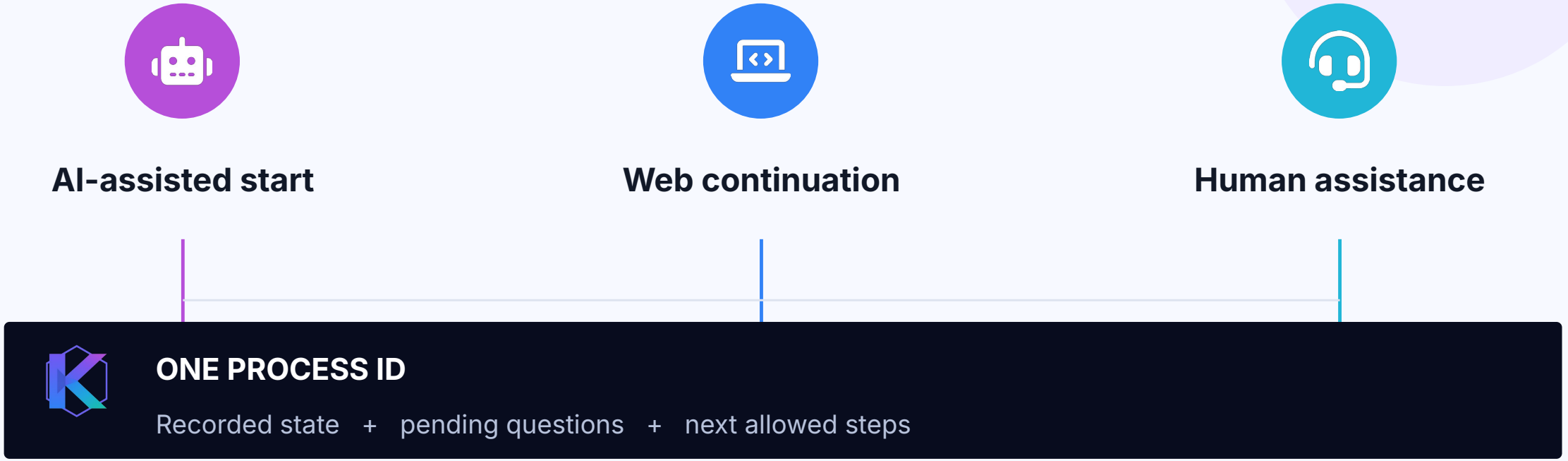
Search reads information. The step performs the action. The process carries the progress.

Illustrative provider-selection interaction based on the sample; not a live run or a fixed workflow sequence.

CONTINUITY ACROSS CONSUMERS

The process should outlive the conversation.

A conversation is an interface. The process has its own identity, recorded state, and continuation contract.



The next consumer reloads the process state—not just the previous chat history.

Supported architectural pattern, not proof of a live handoff. Durable storage, identity, and access control are deployment responsibilities.

A process runtime—not a whole case-management platform.

Use Kaleido for reusable business queries and steps. Keep work management, channels, and external standards at their boundaries.



Workflow & case

Pega and similar platforms

Keep assignments, SLAs, and human work in the workflow platform. Call shared information and actions.

Coordinate work; reuse capabilities.



Voice & assisted

IVR and contact center

Use the same inputs, validation, and process identity across self-service and assisted interactions.

Change channels; retain the model.



Standards & partners

Da Vinci and other boundaries

Map external exchange into stable internal information and action contracts rather than duplicating them.

Adapt the boundary; share the business.

AI is a first-class consumer of that same foundation—not a reason to rebuild it.

Examples of intended integration fit from the original deck; no native Pega connector or Da Vinci conformance claim.

AUTHORITY AND RELEASE POSTURE

AI-native does not mean unbounded.

Use the model for interaction. Keep authorization, business meaning, and execution guarantees explicit.

IMPLEMENTED MECHANISMS

Discoverable contracts

Tool schemas and caller-filtered catalog.

Process constraints

Input structure, step rules, and state.

Guided interaction

Lookups, information requests, and next steps.

HOST + BUSINESS RESPONSIBILITIES

Who may act

Identity, destination credentials, case access.

What the action means

Domain validation and any required approvals.

What is safe to promise

Retry, completion, privacy, and consent tests.

A prompt or tool hint is guidance—not proof of permission, consent, or exactly-once execution.

Preview scope: MCP credential delegation, retry/completion behavior, and real-client consent flows still require validation.

EXECUTIVE TAKEAWAY

Build the business action once. Use it from applications and AI.

> **A .NET runtime for discoverable business information, executable steps, and process continuation— with an AI-native interface.**



01

Define the work

Queries and steps with explicit inputs, relationships, and business handlers.

02

Execute and continue

Run the handler, record progress, and return the next permitted work.

03

Reuse through HTTP + MCP

Different consumers; the same actions. Business authority stays with the host.

Define. Discover. Execute. Continue.